Practical 9

Secure User Registration with Database Insert, Duplicate Email Check, and Password Hashing

Problem Definition: Implement secure user registration by inserting user details into MySQL using MYSQLi. Validate input on frontend and backend, check duplicate email/ username, hash passwords using password_hash(), and display
appropriate messages.

A. Theory

1. Is password_hash() used before storing passwords?

Answer:

Yes, the password_hash() function is used to convert the user's password into a secure hash before storing it in the database. This helps protect passwords from being exposed if the database is compromised.

https://www.geeksforgeeks.org/php/php-password_hash-function/

2. Is duplicate email/username checked?

Answer:

Yes, the system checks whether the email address or username already exists in the database before inserting a new user. This prevents duplicate accounts and helps maintain data consistency.

https://www.geeksforgeeks.org/php/how-to-use-prepared-statements-in-php/?

3. Are frontend and backend validations consistent?

Answer:

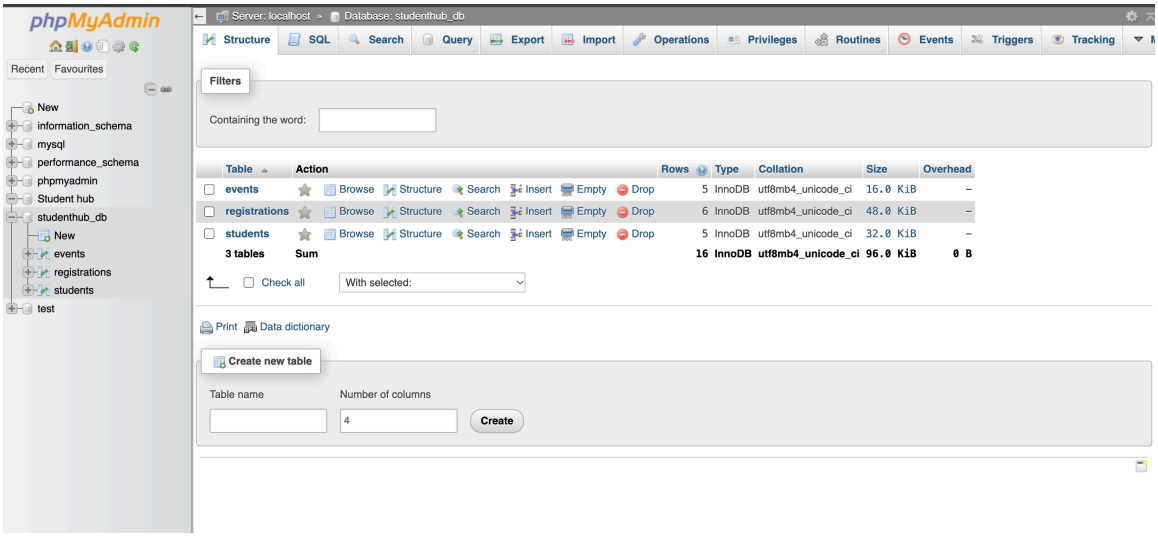
Yes, frontend validation provides immediate feedback to users, while backend validation checks the submitted data before saving it to the database. Both validations should enforce the same rules for required fields, valid email addresses, and password requirements.

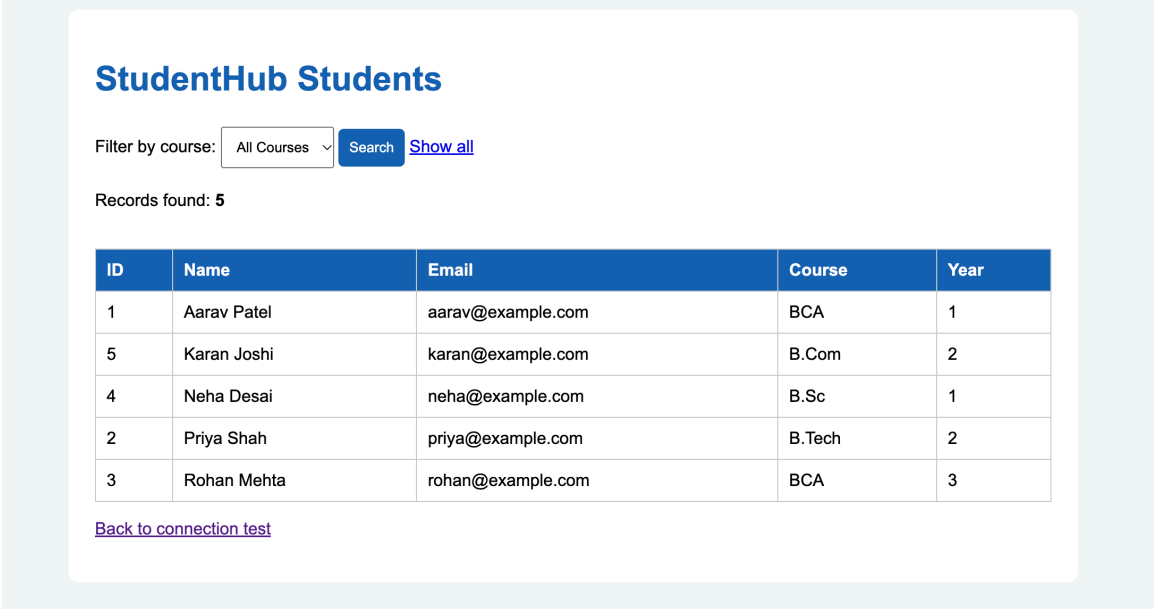
https://www.w3schools.com/php/php_form_validation.asp/

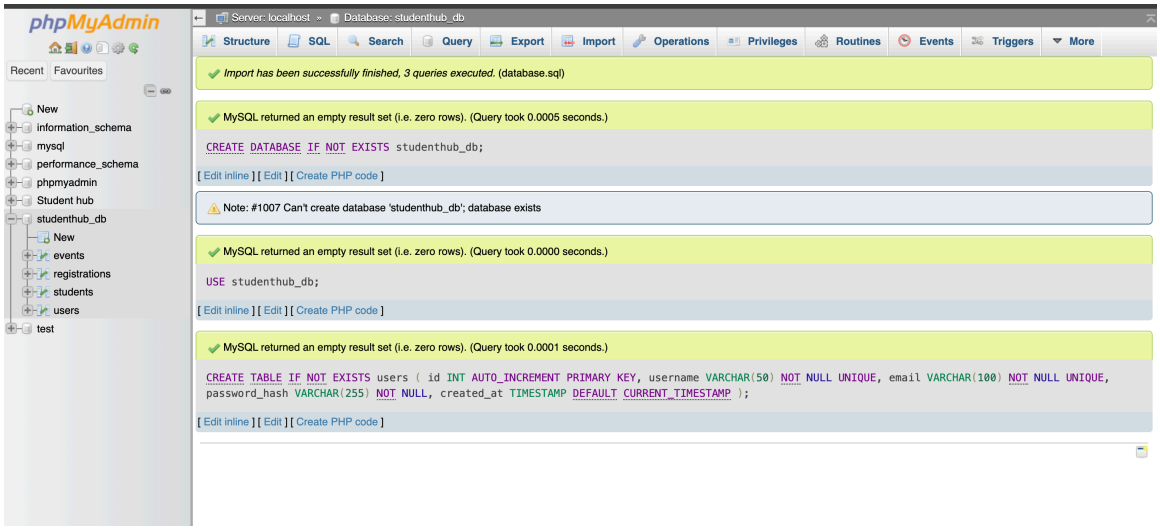
B. Result

Thus, secure user registration was implemented using PHP and MySQLi with input validation, duplicate email and username checks, password hashing, and database insertion to protect user information and maintain data integrity.

C. Evidence of Work (Screenshots)







D. GitHub Repository Link:

https://github.com/Dhanraj2408/StudentHub

E. Challenges Faced:

The main challenges faced were implementing consistent frontend and backend validation, checking for duplicate email addresses and usernames, understanding password hashing, connecting PHP to MySQL, and handling database errors during registration.

F. Learning Outcomes:

The practical helped in understanding secure user registration using PHP and MySQLi, validating user inputs on both frontend and backend, checking duplicate email addresses and usernames, hashing passwords using `password_hash()`, inserting user details into the database, and applying basic web security practices to protect user information.

Faculty Remarks: